



---

# ***EMERGENCY SOS SIMULATOR***

---

A Python-Based Emergency Decision and SOS  
Simulation System – PROJECT REPORT



**Student Name: Nancy Tiwari**  
**Registration no: 26BSA10038**  
**Academic Year: 2026**

## 2. Introduction

### 2.1 Background

Emergencies can occur unexpectedly, and people may need to make quick and appropriate decisions while waiting for professional assistance. In stressful situations, confusion or incorrect actions can increase the risk to individuals. The **Emergency SOS Simulator** is a Python-based console application designed to simulate different emergency situations and guide users through possible responses. The project focuses on decision-making, input validation, emergency information handling, and incident storage.

### 2.2 Project Overview

The system provides different emergency scenarios such as **medical emergencies, fire emergencies, personal safety situations, and SOS activation**. Users interact with the application through menu-based inputs. The project also allows users to provide emergency details such as the **type of emergency, severity, and location**. These details are stored as incident records and can later be viewed through the incident history option.

### 2.3 Purpose

The main purpose of the project is to demonstrate how Python programming concepts can be applied to create a simple emergency simulation system with:

- Modular program structure
- User input and validation
- Emergency scenario handling
- Incident management
- Data storage
- Error handling
- Automated testing

This project is a **simulation and educational tool** and is not intended to replace professional emergency services.

### 3. Problem Statement

During an emergency, people may face difficulty deciding what action to take, especially when they are under stress or lack prior knowledge about appropriate responses. There is a need for a simple system that can simulate emergency situations and present users with different decision-making choices.

The **Emergency SOS Simulator** addresses this problem by providing an interactive, menu-driven Python application in which users can explore emergency scenarios and select possible responses.

The system also addresses the need to record important incident information such as:

- Emergency type
- Severity level
- Location
- Incident history

The project therefore provides a simple way to **simulate emergency decision-making and maintain basic incident records** while demonstrating fundamental Python programming concepts.

#### Problem Definition

To develop a Python-based emergency simulation system that allows users to interact with emergency scenarios, make response decisions, validate their inputs, activate an SOS record, and view previously stored incidents.

## 4. Functional Requirements

The **Emergency SOS Simulator** provides the following major functional requirements:

### **FR-1: Main Menu**

The system shall provide a menu through which the user can access the available functions of the simulator.

### **FR-2: Emergency Scenario Simulation**

The system shall allow users to interact with different emergency scenarios and make decisions based on the situations presented.

The scenarios include:

- Medical emergency
- Fire emergency
- Personal safety
- Other emergency/SOS situations

### **FR-3: SOS Activation**

The system shall allow the user to activate an SOS by entering relevant emergency information. The information includes:

- Emergency type
- Severity
- Location

### **FR-4: Input Validation**

The system shall validate user inputs and display appropriate messages when invalid information is entered. Examples include:

- Invalid menu choices
- Invalid emergency types
- Invalid severity levels
- Empty location input

## **FR-5: Incident Storage**

The system shall store the details of activated SOS incidents for later access.

## **FR-6: Incident History**

The system shall allow users to view previously stored emergency incidents.

## **FR-7: Error Handling**

The system shall handle incorrect or unexpected user inputs without terminating the application unexpectedly.

## **FR-8: Automated Testing**

The project shall include automated tests to verify important system functionality.

## **Major Functional Modules**

The project contains multiple meaningful modules/files, including:

1. **SOS System** — manages SOS-related operations.
2. **Emergency Handler** — handles emergency scenarios and responses.
3. **Incident Manager** — manages incident records.
4. **Response Simulator** — handles emergency decision simulations.
5. **Validator** — validates user inputs.
6. **Configuration** — contains project configuration.
7. **Main Module** — controls the application flow.

## 5. Non-Functional Requirements

The non-functional requirements describe the quality, reliability, and usability expected from the **Emergency SOS Simulator**.

### **NFR-1: Usability**

The application should provide a simple, menu-driven interface so that users can understand the available options and interact with the system easily.

### **NFR-2: Reliability**

The system should continue functioning correctly during normal use and should handle invalid inputs without unexpectedly terminating.

### **NFR-3: Error Handling**

The application should detect invalid or incomplete user inputs and provide appropriate feedback instead of processing incorrect data.

### **NFR-4: Maintainability**

The project should use separate modules for different responsibilities, making the code easier to understand, modify, test, and maintain.

### **NFR-5: Performance**

The application should respond to user inputs quickly because it is a lightweight console-based Python application.

### **NFR-6: Data Integrity**

Incident information should be stored in a structured manner so that previously recorded incidents can be retrieved correctly.

### **NFR-7: Resource Efficiency**

The application should operate with minimal system resources because it does not require heavy external frameworks or computationally intensive processing.

### **NFR-8: Testability**

Important functionality should be testable through automated tests, allowing errors to be identified during development.

## 6. System Architecture

The **Emergency SOS Simulator** follows a modular architecture in which different Python modules are responsible for different parts of the system.

### 6.1 Architecture Overview

The overall flow of the system is:

**User → Main Module → Functional Modules → Data Storage → Output**

The main components are:

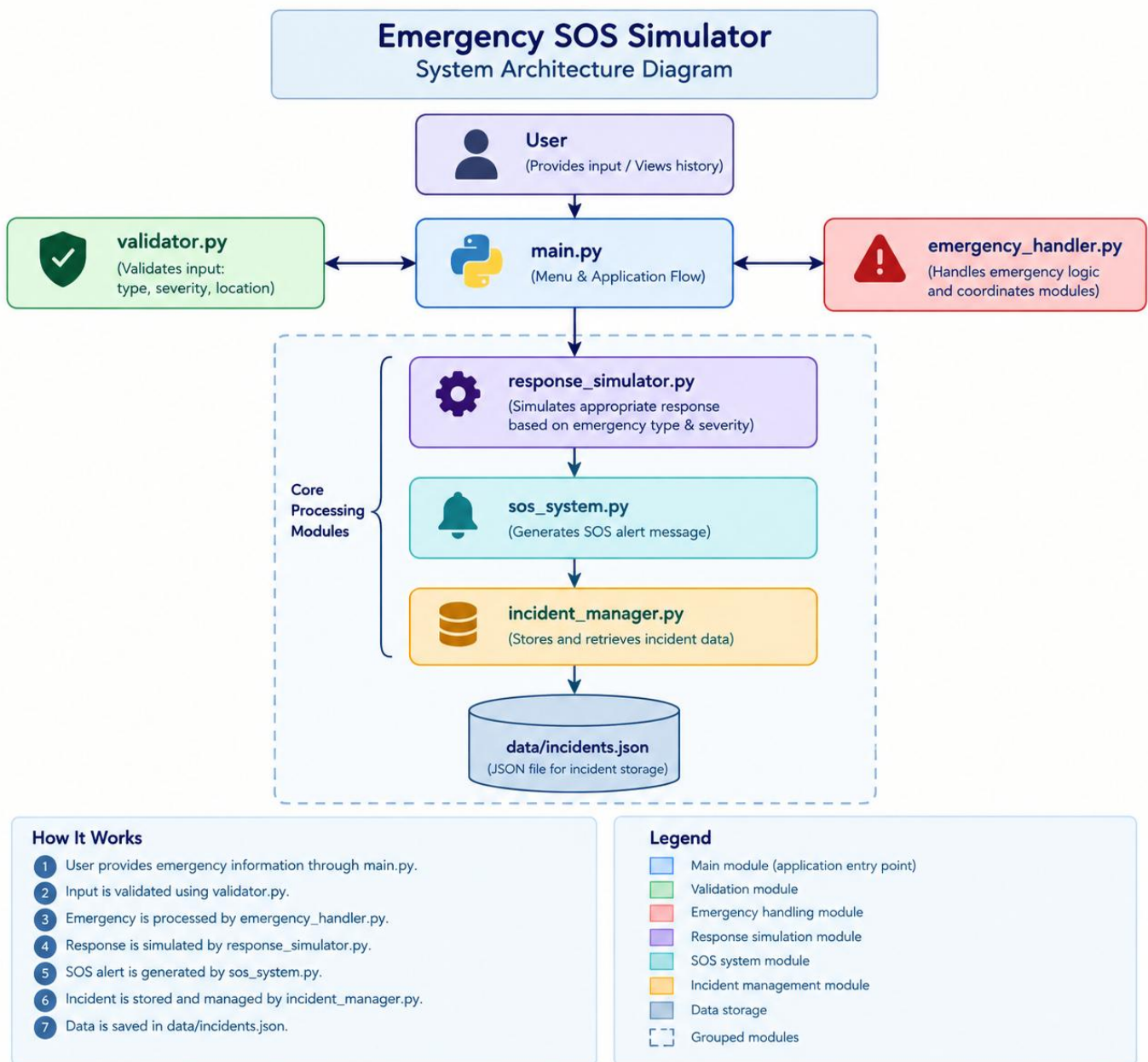
- **main.py** — controls the overall application flow and user interaction.
- **sos\_system.py** — manages SOS-related functionality.
- **emergency\_handler.py** — handles emergency scenarios.
- **response\_simulator.py** — provides interactive emergency decision simulations.
- **validator.py** — validates user inputs.
- **incident\_manager.py** — manages incident records and history.
- **config.py** — contains project configuration.
- **Data Storage** — stores incident information for later retrieval.
- **tests/test\_sos.py** — provides automated testing.

### 6.2 System Workflow

1. The user starts the application.
2. **main.py** displays the main menu.
3. The user selects an available operation.
4. The appropriate module processes the request.
5. User inputs are validated.
6. If an SOS is activated, the incident information is stored.

- The user can later access the incident history.
- The system displays the appropriate result to the user.

### 6.3 Architecture Diagram



## 7. Design Diagrams

The project includes several design diagrams that represent the structure, workflow, and interaction of the Emergency SOS Simulator.

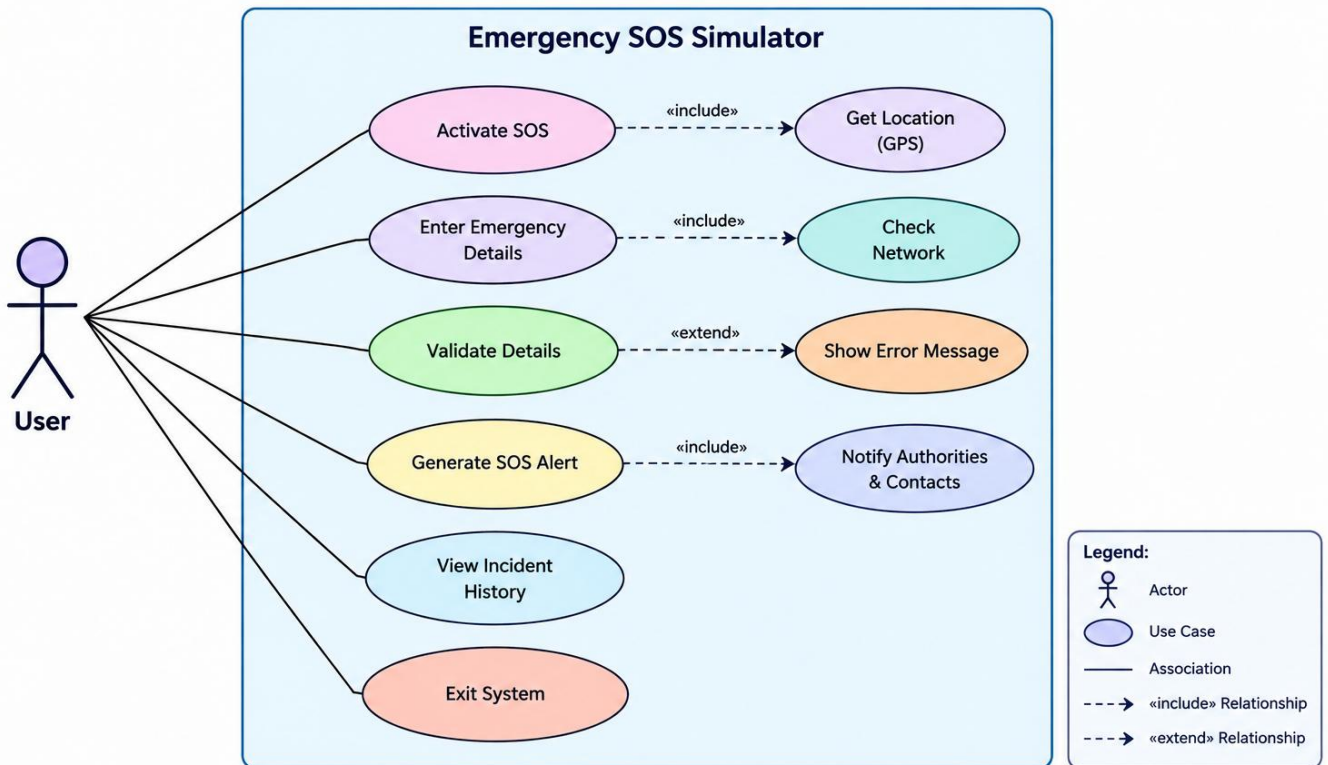
### 7.1 Use Case Diagram

The Use Case Diagram represents the interaction between the user and the major functions of the system.

The user can:

- **Start the simulator**
- **Select an emergency scenario**
- **Make decisions**
- **Activate SOS**
- **Enter emergency details**
- **View incident history**
- **Exit the application**

## Emergency SOS Simulator – Use Case Diagram

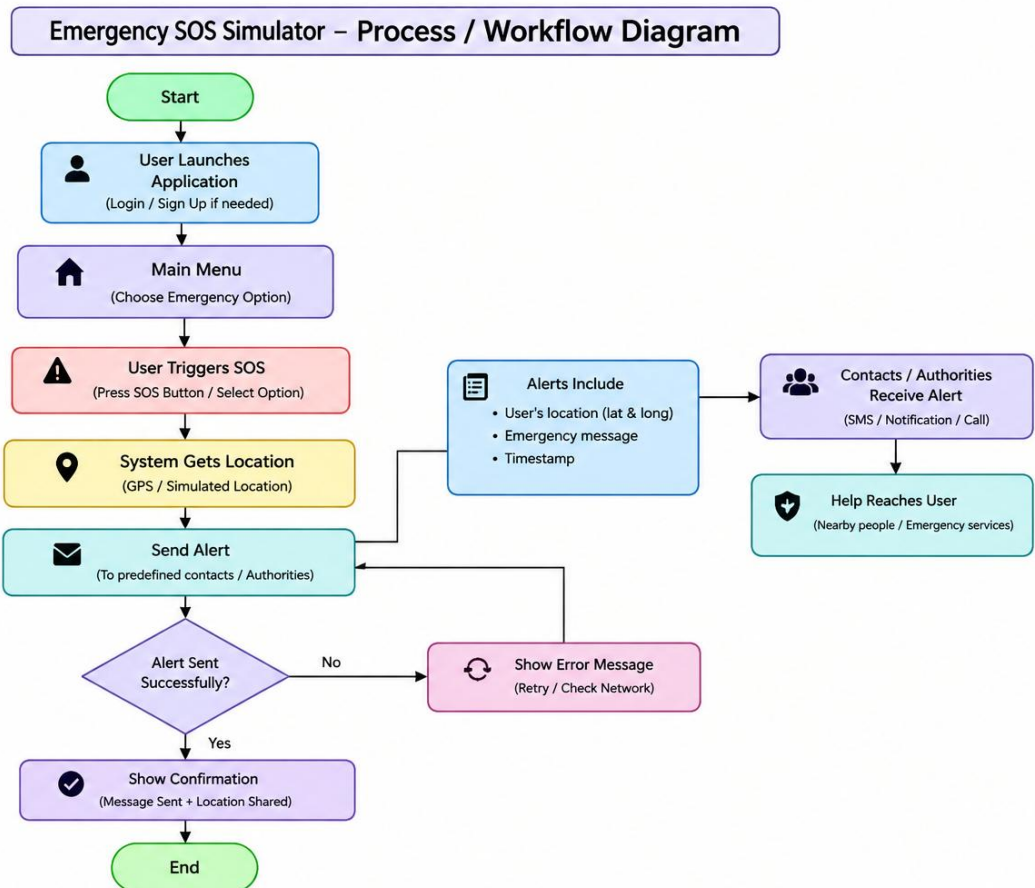


## 7.2 Process Workflow Diagram

The Process Workflow Diagram represents the sequence of steps followed by the system from starting the application to completing an operation.

It shows:

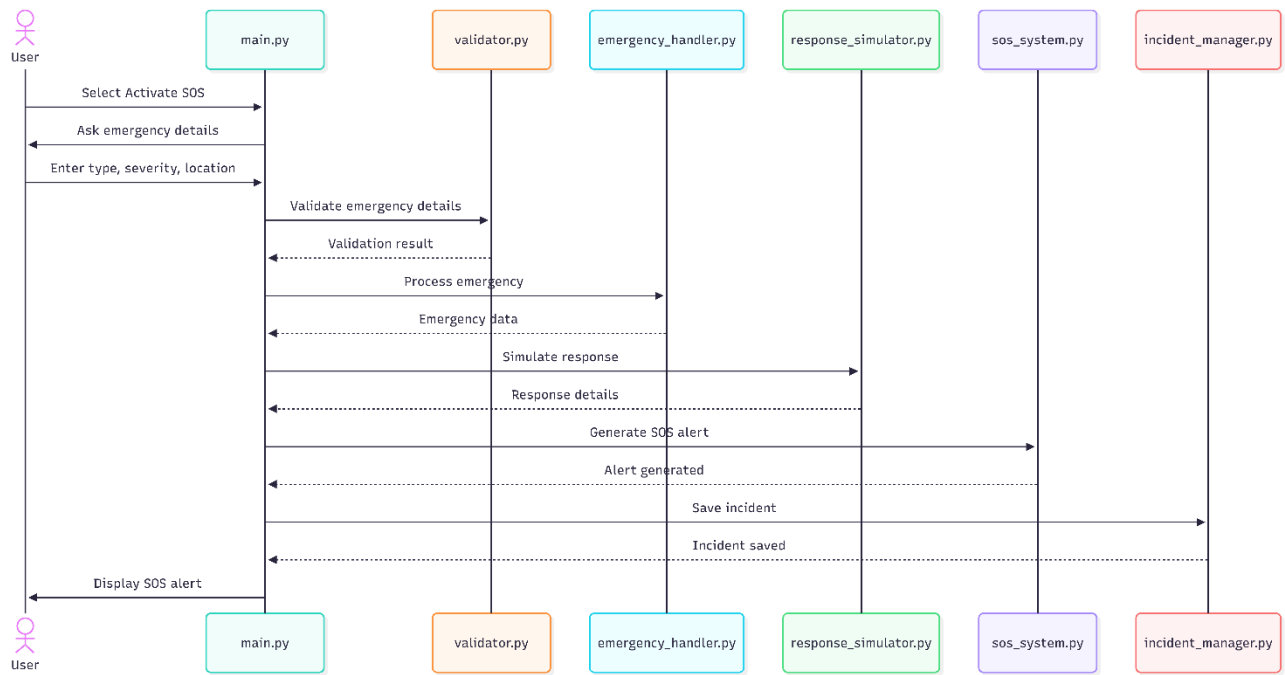
- Starting the application
- Displaying the menu
- Selecting an operation
- Processing user input
- Validating information
- Saving or displaying incident information
- Returning to the menu or exiting



## 7.3 Sequence Diagram

The Sequence Diagram illustrates the interaction between the user and the different system components during an operation.

It represents how requests move through the application and how the system responds to the user.

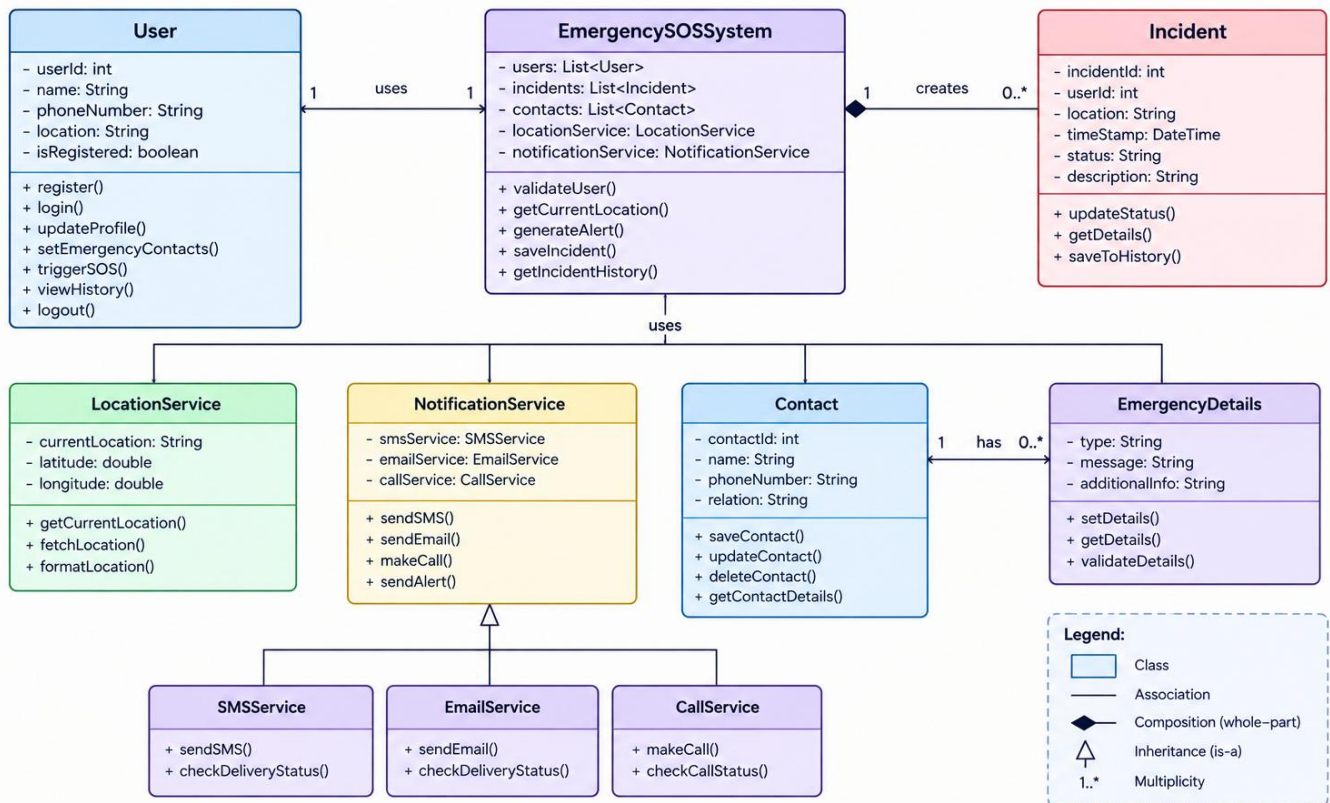


## 7.4 Class Diagram

The Class Diagram represents the logical structure of the project's components and their relationships.

It helps explain how the different modules/classes involved in the application are organized.

## Emergency SOS Simulator – Class Diagram



## 7.5 Storage Design

The project also uses structured storage for maintaining incident information. The storage design describes how emergency incident records are organized and retrieved.

## **# Storage Design**

### **## Storage Method**

The Emergency SOS Simulator uses a JSON file for storing incident history.

### **## Storage Location**

All incidents records are stored in:

``data/incidents.json``

### **## Stored Information**

Each incident record contains:

- Emergency Type
- Severity
- Location
- Response Unit
- Response Time
- Status

### **## Saving Data**

The ``incident_manager.py`` module saves each completed SOS incident to the JSON file.

When a new incident is created, it is added to the existing incident records.

### **## Retrieving Data**

The ``incident_manager.py`` module reads the JSON file whenever the user selects **\*\*View Incident History\*\***.

The stored incidents are then displayed through the main program.

## **## Data Format**

The information is stored as a list of JSON objects, making it simple to read, update, and maintain.

## **## Error Handling**

If the storage files does not exist, the system starts with an empty incident history. If the JSON file contains invalid data, the system safely handles the error and returns an empty incident list.

## **Diagram Summary**

| <b>Diagram</b>   | <b>Purpose</b>                 |
|------------------|--------------------------------|
| Use Case Diagram | Shows user-system interactions |
| Process Workflow | Shows the overall process flow |
| Sequence Diagram | Shows interaction sequence     |
| Class Diagram    | Shows system structure         |
| Storage Design   | Explains incident data storage |

## 8. Design Decisions & Rationale

The project was designed as a **modular, console-based Python application** so that each major responsibility can be handled separately and understood easily.

### 8.1 Modular Structure

The application is divided into multiple Python files, with each module handling a specific responsibility.

**Reason:**

A modular structure makes the project easier to understand, test, maintain, and modify.

### 8.2 Console-Based Interface

The project uses a menu-driven console interface rather than a graphical interface.

**Reason:**

A console interface keeps the project lightweight and allows the focus to remain on Python programming concepts, logic, validation, and emergency simulation.

### 8.3 Scenario-Based Simulation

Different emergency situations are presented as interactive scenarios where the user selects possible responses.

**Reason:**

This makes the project more interactive than a simple information-display program and demonstrates conditional logic and user-driven program flow.

### 8.4 Input Validation

The system validates important user inputs such as menu choices, emergency types, severity, and location.

**Reason:**

Validation prevents incorrect or incomplete information from being processed and improves the reliability of the application.

## 8.5 Incident Storage

Emergency incidents are stored so that they can be accessed later through the incident-history functionality.

**Reason:**

Storing incidents demonstrates how a Python application can maintain information beyond a single interaction.

## 8.6 Automated Testing

The project includes a separate tests folder containing automated tests.

**Reason:**

Testing helps verify important functionality and provides a repeatable way to check the application after changes.

## 8.7 Version Control

The project is maintained in a GitHub repository with organized source files, tests, documentation, and diagrams.

**Reason:**

Version control provides an organized way to maintain the project and satisfies the project's technical documentation requirements.

## 8.8 Educational Scope

The application is designed as a **simulation and educational project**, not as a replacement for real emergency-response systems.

**Reason:**

Real emergency systems require specialized infrastructure, verified emergency-service integration, security controls, and extensive testing beyond the scope of a beginner-level Python project.

## 9. Implementation Details

The **Emergency SOS Simulator** was implemented using Python with a modular structure. Each module performs a specific task, making the application easier to understand, test, and maintain.

### 9.1 Main Module — `main.py`

`main.py` acts as the entry point of the application. It:

- Displays the main menu.
- Accepts the user's choice.
- Connects the user with different emergency functions.
- Handles the overall program flow.
- Allows the user to exit the application.

### 9.2 SOS System — `sos_system.py`

This module manages the SOS functionality. It:

- Collects emergency information.
- Processes the SOS request.
- Connects with incident-management functionality.
- Provides the appropriate output to the user.

### 9.3 Emergency Handler — `emergency_handler.py`

This module handles different emergency scenarios such as:

- Medical emergencies
- Fire emergencies
- Personal safety situations
- Other emergency situations

It uses decision-making logic to provide suitable responses according to the selected scenario.

## **9.4 Response Simulator — `response_simulator.py`**

This module is responsible for simulating possible emergency responses. It provides the user with response options and processes their selections.

## **9.5 Validator — `validator.py`**

The validator module checks whether the information entered by the user is valid. It helps validate:

- Menu selections
- Emergency types
- Severity levels
- Location information

This reduces the possibility of invalid data being processed.

## **9.6 Incident Manager — `incident_manager.py`**

The Incident Manager handles emergency incident records. Its responsibilities include:

- Saving incident information.
- Retrieving previously stored incidents.
- Supporting the incident-history feature.

## **9.7 Configuration — `config.py`**

`config.py` contains configuration-related values used by different parts of the project. Keeping configuration separately helps avoid unnecessary duplication across modules.

## 9.8 Automated Tests — tests/test\_sos.py

The project includes automated tests for important functionality. The tests help verify that the application behaves correctly for different inputs and situations.

## 9.9 Documentation Files

The project also contains supporting documentation:

- **README.md** — project overview, features, running instructions, and testing instructions.
- **statement.md** — project problem statement and scope.
- **storage\_design.md** — storage-related design information.
- **diagrams/** — system and design diagrams.
- 

## Implementation Summary

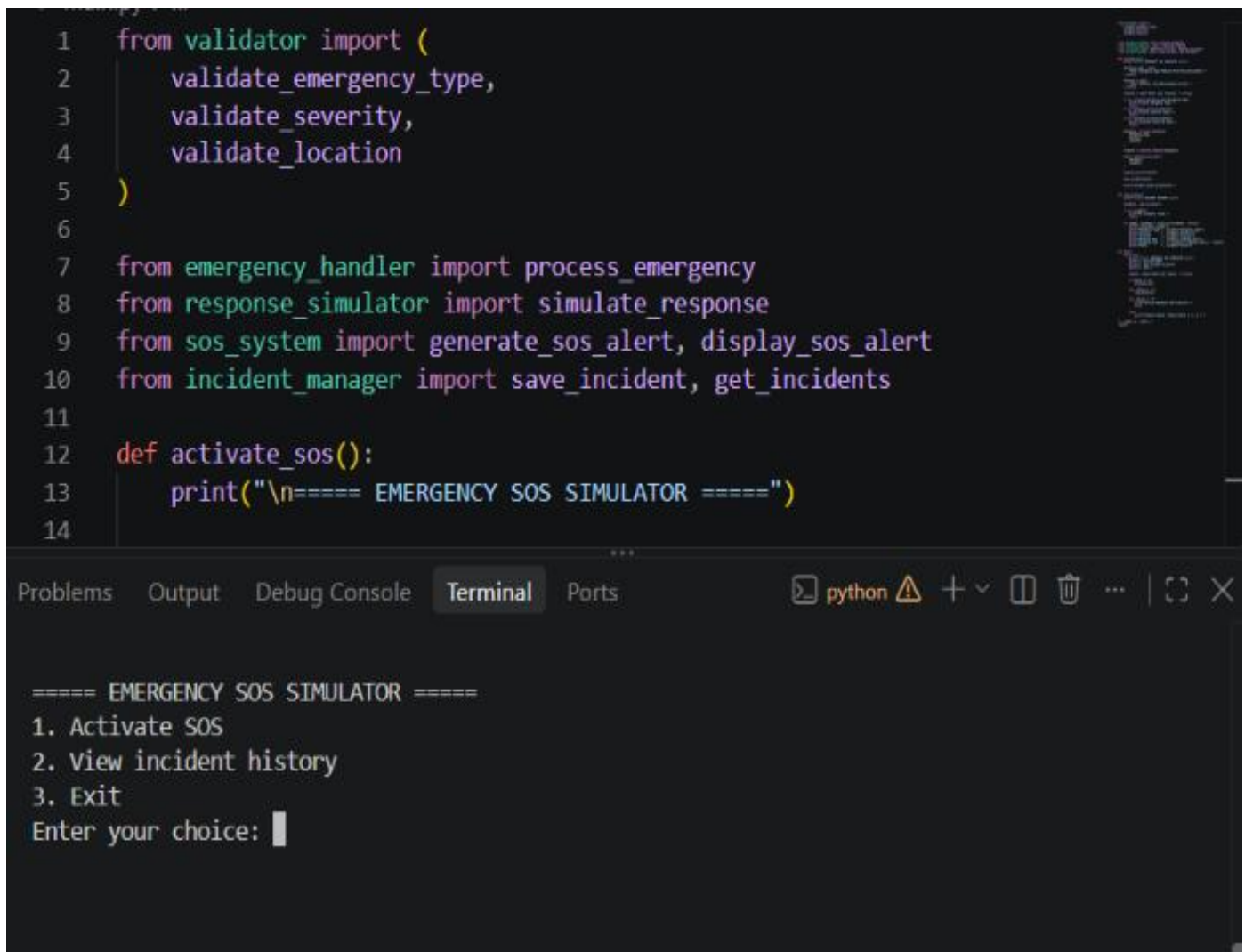
| Module/File           | Main Responsibility      |
|-----------------------|--------------------------|
| main.py               | Main program flow        |
| sos_system.py         | SOS functionality        |
| emergency_handler.py  | Emergency scenarios      |
| response_simulator.py | Response simulation      |
| validator.py          | Input validation         |
| incident_manager.py   | Incident storage/history |
| config.py             | Configuration            |
| tests/test_sos.py     | Automated testing        |

## 10. Screenshots / Results

The project was executed and tested in VS Code to verify its main functionalities. The following screenshots demonstrate the working of the Emergency SOS Simulator.

### 10.1 Main Menu

The main menu provides the user with different options to interact with the Emergency SOS Simulator.



The screenshot displays the Visual Studio Code interface. The top editor pane shows a Python script with the following code:

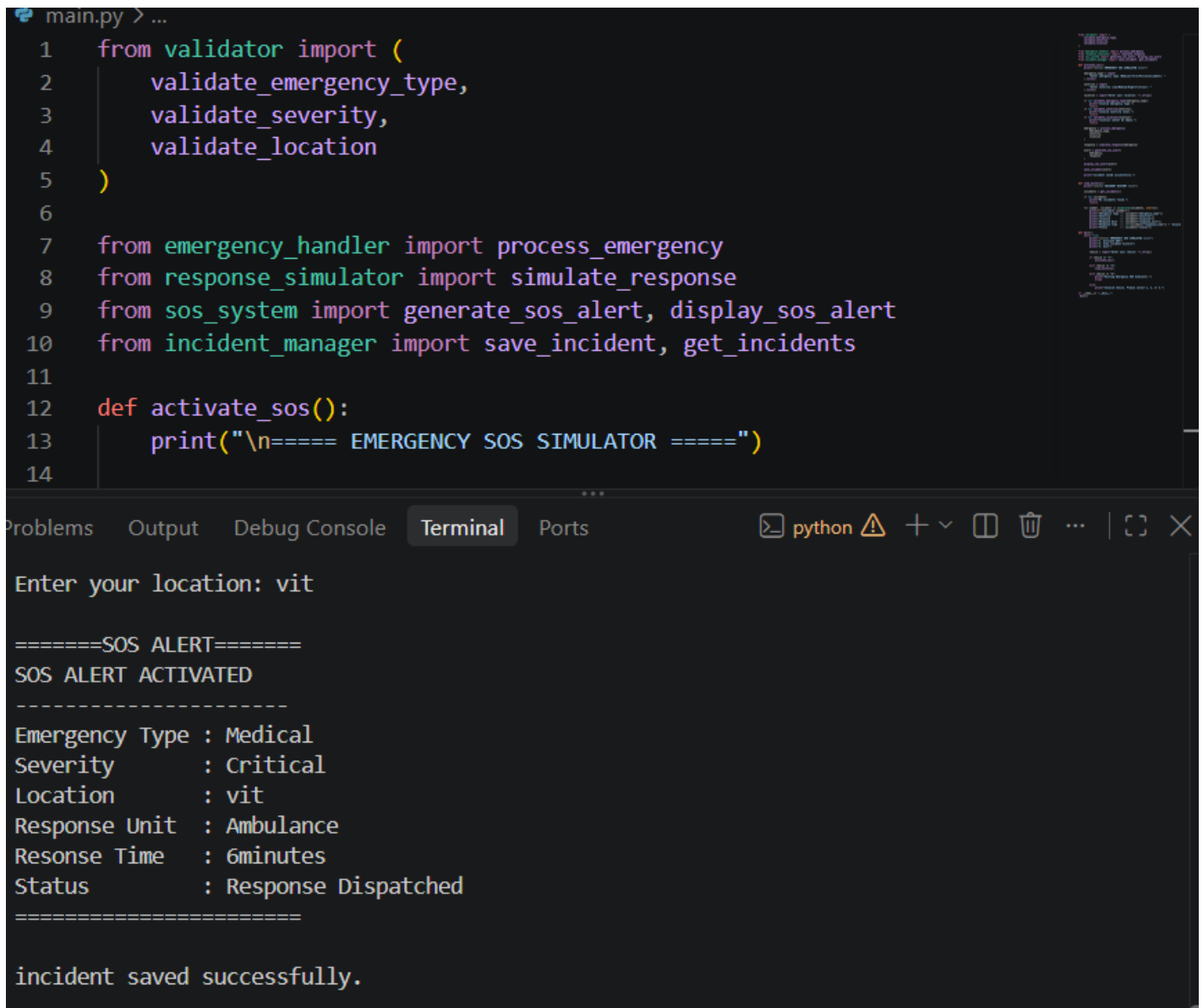
```
1  from validator import (  
2      validate_emergency_type,  
3      validate_severity,  
4      validate_location  
5  )  
6  
7  from emergency_handler import process_emergency  
8  from response_simulator import simulate_response  
9  from sos_system import generate_sos_alert, display_sos_alert  
10 from incident_manager import save_incident, get_incidents  
11  
12 def activate_sos():  
13     print("\n===== EMERGENCY SOS SIMULATOR =====")  
14
```

The bottom terminal pane shows the output of the script, which is the main menu of the Emergency SOS Simulator:

```
===== EMERGENCY SOS SIMULATOR =====  
1. Activate SOS  
2. View incident history  
3. Exit  
Enter your choice: █
```

## 10.2 Successful SOS Activation

This screenshot shows a successful SOS activation after the user provides the required emergency information such as emergency type, severity, and location.



The screenshot displays a VS Code editor with a Python script in `main.py` and its execution output in the integrated terminal. The script defines an `activate_sos()` function that prints a separator, displays emergency details, and saves the incident. The terminal shows the user inputting 'vit' for the location, followed by a formatted SOS alert and a success message.

```
main.py > ...
1  from validator import (
2      validate_emergency_type,
3      validate_severity,
4      validate_location
5  )
6
7  from emergency_handler import process_emergency
8  from response_simulator import simulate_response
9  from sos_system import generate_sos_alert, display_sos_alert
10 from incident_manager import save_incident, get_incidents
11
12 def activate_sos():
13     print("\n===== EMERGENCY SOS SIMULATOR =====")
14
```

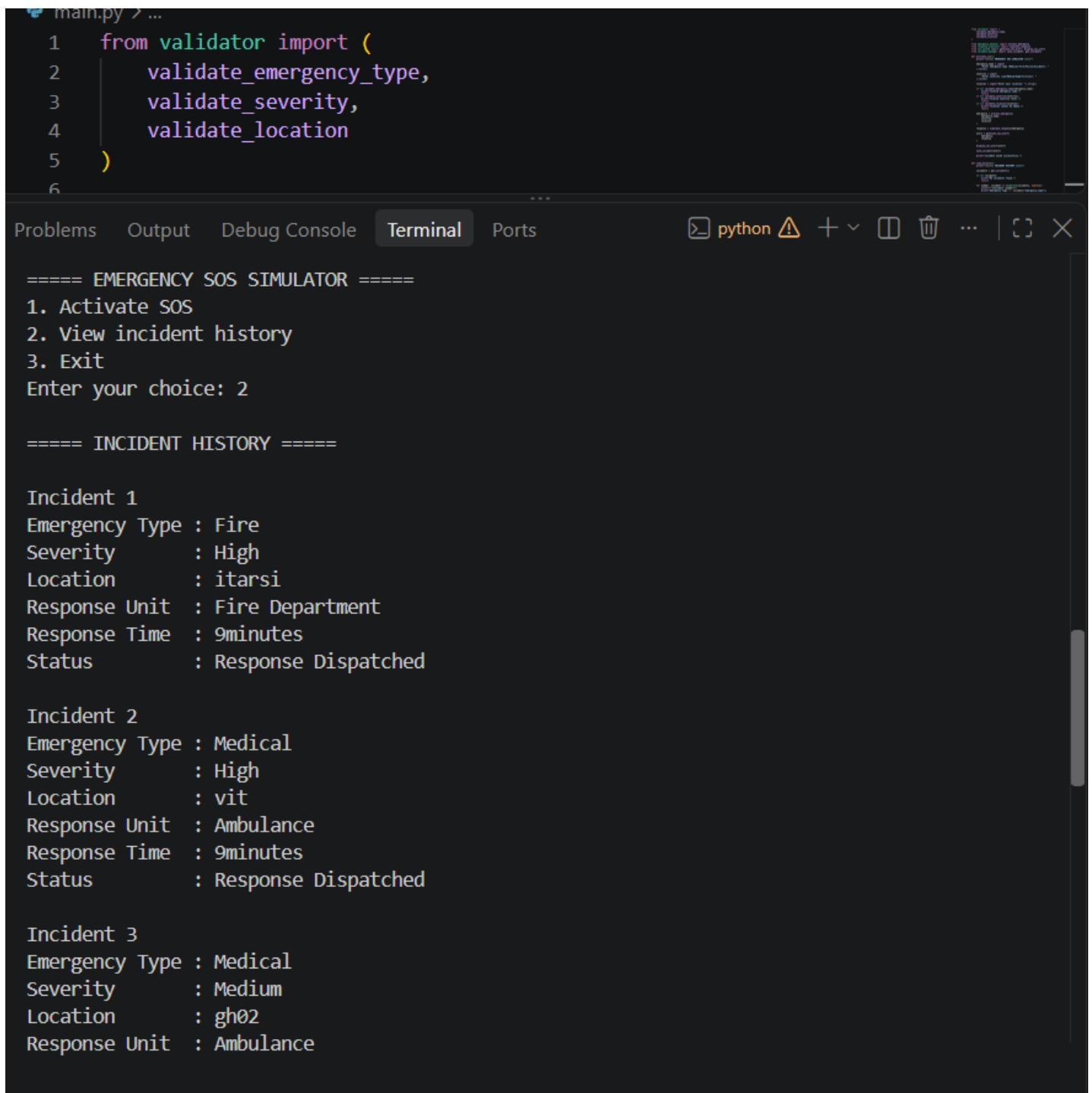
Enter your location: vit

```
=====SOS ALERT=====
SOS ALERT ACTIVATED
-----
Emergency Type : Medical
Severity       : Critical
Location      : vit
Response Unit  : Ambulance
Resonse Time   : 6minutes
Status        : Response Dispatched
=====

incident saved successfully.
```

## 10.3 Incident History

This screenshot demonstrates that the emergency incident was successfully stored and can be viewed through the incident-history option.



The screenshot shows a VS Code editor with a Python file named `main.py` and a terminal window. The Python code in `main.py` imports validation functions from a `validator` module. The terminal output shows the execution of an emergency simulator program. The user selects option 2 to view incident history, and the program displays three incidents with their details.

```
main.py > ...
1  from validator import (
2      validate_emergency_type,
3      validate_severity,
4      validate_location
5  )
6

===== EMERGENCY SOS SIMULATOR =====
1. Activate SOS
2. View incident history
3. Exit
Enter your choice: 2

===== INCIDENT HISTORY =====

Incident 1
Emergency Type : Fire
Severity       : High
Location      : itarsi
Response Unit  : Fire Department
Response Time  : 9minutes
Status        : Response Dispatched

Incident 2
Emergency Type : Medical
Severity       : High
Location      : vit
Response Unit  : Ambulance
Response Time  : 9minutes
Status        : Response Dispatched

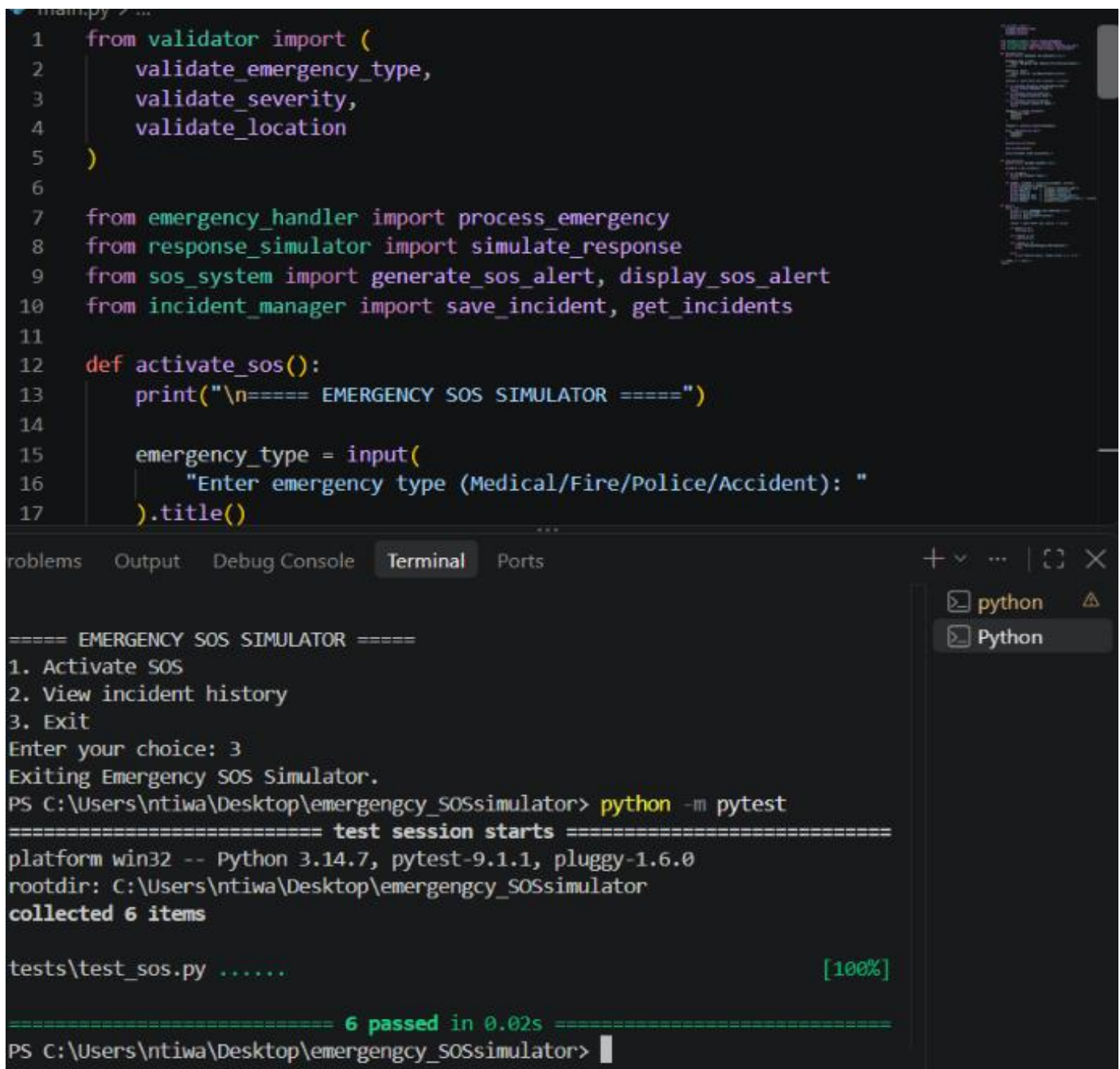
Incident 3
Emergency Type : Medical
Severity       : Medium
Location      : gh02
Response Unit  : Ambulance
```

## 10.4 Automated Testing

Automated testing was performed using:

```
python -m pytest
```

The test execution completed successfully, confirming that the implemented test cases passed.



```
1  from validator import (
2      validate_emergency_type,
3      validate_severity,
4      validate_location
5  )
6
7  from emergency_handler import process_emergency
8  from response_simulator import simulate_response
9  from sos_system import generate_sos_alert, display_sos_alert
10 from incident_manager import save_incident, get_incidents
11
12 def activate_sos():
13     print("\n===== EMERGENCY SOS SIMULATOR =====")
14
15     emergency_type = input(
16         "Enter emergency type (Medical/Fire/Police/Accident): "
17     ).title()
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
==== EMERGENCY SOS SIMULATOR ====
1. Activate SOS
2. View incident history
3. Exit
Enter your choice: 3
Exiting Emergency SOS Simulator.
PS C:\Users\ntiwa\Desktop\emergengcy_SOSsimulator> python -m pytest
===== test session starts =====
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\ntiwa\Desktop\emergengcy_SOSsimulator
collected 6 items

tests\test_sos.py ..... [100%]

===== 6 passed in 0.02s =====
PS C:\Users\ntiwa\Desktop\emergengcy_SOSsimulator>
```

## 10.5 Results Summary

The testing and screenshots demonstrate that the project successfully supports:

- Menu-driven interaction
- Emergency scenario selection
- SOS activation
- Input validation
- Incident storage
- Incident history
- Automated testing
- Proper program exit and error handling

These results demonstrate the practical working of the implemented system.

## 11. Testing Approach

Testing was performed to verify that the Emergency SOS Simulator works correctly under both valid and invalid user inputs.

### 11.1 Manual Testing

The application was manually executed through the VS Code terminal. Different options and input conditions were tested to check the application's behavior.

The following cases were tested:

| Test Case | Input/Action                  | Expected Result                        |
|-----------|-------------------------------|----------------------------------------|
| TC-01     | Start application             | Main menu is displayed                 |
| TC-02     | Select valid emergency option | Appropriate scenario is opened         |
| TC-03     | Enter valid SOS details       | SOS is activated and incident is saved |
| TC-04     | View incident history         | Stored incident is displayed           |
| TC-05     | Enter invalid menu choice     | Error message is displayed             |
| TC-06     | Enter invalid emergency type  | Input is rejected                      |
| TC-07     | Enter invalid severity        | Input is rejected                      |
| TC-08     | Leave location empty          | Validation error is displayed          |
| TC-09     | Select Exit                   | Application terminates correctly       |

## 11.2 Automated Testing

Automated tests were implemented in:

```
tests/test_sos.py
```

The tests were executed using:

```
python -m pytest
```

The test execution completed successfully, as shown in the test-results screenshot in Section 10.

## 11.3 Error and Validation Testing

Special attention was given to invalid inputs. The application was tested with incorrect menu choices, invalid emergency types, invalid severity values, and empty location input.

This helped verify that invalid information is handled instead of being processed as a valid emergency record.

## 11.4 Testing Outcome

Testing confirmed that the major functionalities of the application—including SOS activation, incident storage, incident history, validation, error handling, and program termination—work as intended.

## 12. Challenges Faced

During the development of the **Emergency SOS Simulator**, several challenges were encountered while designing, implementing, testing, and documenting the project.

### 12.1 Understanding Project Structure

Initially, organizing the project into multiple Python files and folders was challenging.

**Solution:**

The project was divided into separate modules based on their responsibilities, such as validation, SOS handling, incident management, and response simulation.

### 12.2 Implementing Input Validation

Handling different types of user input was another challenge. Incorrect menu choices, emergency types, severity levels, and empty locations needed to be handled properly.

**Solution:**

A separate validator.py module was created to validate user inputs and display appropriate error messages.

### 12.3 Managing Incident Data

The application needed to save emergency incidents and retrieve them later through the incident-history feature.

**Solution:**

An incident\_manager.py module was used to handle incident storage and retrieval.

### 12.4 Debugging and Testing

While developing the application, errors could occur when different modules interacted with each other.

**Solution:**

The application was tested repeatedly using different valid and invalid inputs. Automated tests were also created in tests/test\_sos.py.

## 12.5 Organizing the GitHub Repository

Maintaining source files, tests, diagrams, and documentation in a proper GitHub structure required additional effort.

### **Solution:**

The repository was organized into separate folders such as tests/ and diagrams/, while documentation files and Python modules were maintained in the project root.

## 12.6 Preparing Project Documentation

Creating the required diagrams, screenshots, README, and project report alongside the actual implementation was another challenge.

### **Solution:**

The documentation was prepared systematically along with screenshots of the working application and the required design diagrams.

## **Overall**

These challenges helped improve understanding of **Python modular programming, debugging, input validation, testing, project organization, and documentation.**

## 13. Learnings & Key Takeaways

Developing the **Emergency SOS Simulator** provided practical experience in Python programming as well as software project development.

### 13.1 Python Programming

The project improved understanding of important Python concepts such as:

- Functions
- Conditional statements
- Loops
- Modules
- Exception handling
- File/data handling

### 13.2 Modular Programming

Dividing the application into separate modules helped demonstrate how a larger program can be organized into smaller, manageable components.

### 13.3 Input Validation

The project provided practical understanding of why user input should be validated before it is processed.

### 13.4 File and Data Management

Working with incident records helped develop an understanding of storing, retrieving, and managing application data.

### 13.5 Testing and Debugging

Testing different scenarios helped identify and correct errors. Automated testing also provided experience in checking functionality systematically.

### 13.6 Project Organization

The project improved understanding of how source code, tests, diagrams, documentation, and other project resources can be organized in a structured repository.

### 13.7 GitHub and Version Control

Using GitHub provided practical experience in maintaining a project repository, organizing files, and documenting project development.

### 13.8 Documentation

Preparing the diagrams, README, screenshots, and project report demonstrated the importance of documenting both the design and implementation of a software project.

### Key Takeaway

The project helped connect **Python programming concepts with a complete software-development workflow**, from problem identification and design to implementation, testing, documentation, and presentation.

## 14. Future Enhancements

Although the current version of the **Emergency SOS Simulator** provides the core simulation and incident-management features, several improvements could be added in future versions.

### 14.1 Graphical User Interface

The console-based application could be converted into a GUI application using technologies such as **Tkinter** or another Python GUI framework.

This would make interaction more visual and user-friendly.

### 14.2 Real-Time Location Support

A future version could support GPS/location services to automatically obtain the user's location during an emergency.

### 14.3 Emergency Service Integration

The system could potentially be integrated with appropriate emergency-service APIs to provide real-world assistance. Such integration would require proper authentication, security, reliability, and authorization.

### 14.4 Mobile Application

The project could be developed as a mobile application so that emergency features can be accessed more conveniently from a smartphone.

### 14.5 Improved Data Management

The current storage approach could be extended to a proper database system for managing a larger number of incident records efficiently.

## **14.6 User Profiles**

Future versions could provide user profiles containing information such as emergency contacts and other voluntarily provided details.

## **14.7 Notification System**

Notifications could be added to inform designated contacts about an activated emergency simulation.

## **14.8 More Emergency Scenarios**

Additional scenarios could be introduced, such as:

- Natural disasters
- Road accidents
- Missing-person situations
- Cyber or digital safety incidents

## **14.9 Advanced Decision Simulation**

The simulator could include more complex decision trees where different user choices lead to different outcomes and response paths.

## **Future Vision**

The project can be gradually extended from a beginner-level Python console simulator into a more comprehensive emergency-assistance simulation platform while maintaining appropriate safety, privacy, and reliability considerations.

## 15. References

The following resources were used during the development, implementation, testing, and documentation of the **Emergency SOS Simulator** project.

1. **VITyarthi – Build Your Own Project Guidelines**

Used for understanding the project requirements, documentation structure, design diagrams, testing requirements, and evaluation criteria.

2. **Python Documentation**

Used as a reference for Python programming concepts, syntax, functions, modules, file handling, and exception handling.

3. **Pytest Documentation**

Used as a reference for implementing and running automated tests for the project.

4. **GitHub Documentation**

Used as a reference for repository organization, file management, and project documentation.

5. **Project Source Code and Design Documents**

The project's own Python modules, README.md, statement.md, storage\_design.md, test files, and design diagrams were used during implementation and documentation.